# OpenMC Pin-Cell Demonstrator: Research Findings, Notebook, environment.yml and README

The best approach is to install **OpenMC 0.16.0** (the current stable release, out since August 5, 2026) from conda-forge with **Python ≥ 3.12**, download the official **ENDF/B-VIII.0 HDF5 library** from openmc.org/data, and build the model as a single reflective $UO_2$/Zircaloy/borated-water pin cell using the `openmc.Model` container. The pandas-dataframes notebook is a useful template for post-processing, but its model-building code is out of date in several places.

## TL;DR

- **Status of the official notebook.** The pandas-dataframes notebook still runs its old workflow: separate `export_to_xml()` files, then `openmc.run()`. Its stored output was produced with OpenMC 0.13.1. It already uses `openmc.IndependentSource`, but in 0.15.0+ two of its calls are deprecated (`openmc.stats.Box(..., only_fissionable=True)` and the legacy `openmc.Plot`), and it predates the `openmc.Model` workflow.
- **Installation.** Run `conda create -n openmc-env -c conda-forge openmc python=3.12`. OpenMC 0.16.0 dropped Python 3.11, so Python 3.12–3.14 is required. Point `OPENMC_CROSS_SECTIONS` (or `openmc.config['cross_sections']`) at the `cross_sections.xml` inside the downloaded library.
- **Deliverables.** The notebook, environment.yml and README below use only API calls documented for the stable release: `openmc.Model`, `IndependentSource(constraints={'fissionable': True})`, `RectangularPrism(boundary_type='reflective')`, `StatePoint` as a context manager, and `Tally.get_pandas_dataframe()`.

---

## Key Findings

### 1. Current version and installation

- **Release.** OpenMC 0.16.0 was released on 5 Aug 2026 by Paul Romano. (github) Its release notes say: "Python 3.11 is no longer supported; OpenMC now requires Python 3.12 or newer and adds support for Python 3.14." The documented minimum build environment is GCC 11.4 and CMake 3.22. (github) (github)
- **conda-forge builds.** The `openmc` package lists 0.16.0 as its latest version, built for linux-64, macOS-64, macOS-arm64 and linux-aarch64. (anaconda)

- **Official Quick Install Guide.** The steps are:
  1. `conda config --add channels conda-forge`
  2. `conda config --set channel_priority strict`
  3. `conda create --name openmc-env openmc` (openmc)
- **Apple silicon.** The guide still tells Apple-silicon users to add `--platform osx-64`. That note is probably stale now that a native osx-arm64 build exists. (anaconda) (openmc) Try the native build first and fall back to `--platform osx-64` only if the solver fails.
- **Windows.** There is no native Windows build. Use WSL2 or Docker (`docker run openmc/openmc:latest`). (openmc)

**Deprecations in 0.16.0 that matter here:**

- The `Plot` compatibility constructor is deprecated. (github) Use `SlicePlot` or `VoxelPlot`. (github)
- `MeshBase.num_mesh_cells` is replaced by `MeshBase.n_elements`. (github) (github)
- New: tally scores and filters can be passed directly to the `Tally` constructor. (github)
- New: nuclides, elements, density, temperature and volume can be passed to the `Material` constructor. (github)

The notebook below sticks to long-standing attribute assignment so that it stays portable.

## 2. Cross-section data

- **How OpenMC finds data.** It reads a `cross_sections.xml` index file that points to HDF5 files. You can set it in Python with `openmc.config['cross_sections']`, which "at startup … is initialized with the value of the `OPENMC_CROSS_SECTIONS` environment variable". `openmc.Materials.cross_sections` overrides both. (openmc)
- **Official libraries on openmc.org/data.** All are `.tar.xz` archives:
  - ENDF/B-VII.1 (NJOY 2016.68)
  - ENDF/B-VIII.0 (NJOY 2016.68)
  - ENDF/B-VIII.1 (NJOY 2016.78) (openmc)
  - JEFF 3.3, JEFF 4.0 and JENDL-5 (openmc)
- **Temperatures.** Incident-neutron data is provided at 250, 293.6, 600, 900, 1200 and 2500 K. (github) Thermal scattering data is provided at the tabulated

temperatures. openmc

- **Recommendation.** Use ENDF/B-VIII.0, which is the most widely used modern baseline. ENDF/B-VIII.1 is newer and also fine. Record the library you used in the README, because k-eff shifts by a few hundred pcm between libraries.
- **Scripted alternative.** The community `openmc_data` package (PyPI 2.4.x) provides `download_endf` for ENDF/B VII.1, VIII.0 and VIII.1. pypi +2 For example, `download_endf -r b8.0` fetches pre-processed HDF5 files. This is third-party tooling maintained outside openmc-dev, so pin the version you use.

### 3. What the pandas-dataframes notebook does (openmc-notebooks, `main`)

| Aspect | Exact content |
|---|---|
| Materials | **"1.6% Fuel"**: 10.31341 g/cm³; U235 3.7503e-4, U238 2.2625e-2, O16 4.6007e-2. **Borated water**: 0.740582 g/cm³; H1 4.9457e-2, O16 2.4732e-2, B10 8.0042e-6. **Zircaloy**: 6.55 g/cm³; Zr90 only, 7.2758e-3. githubusercontent No S(α,β) thermal scattering on the water. |
| Geometry | Pin universe: fuel `ZCylinder(r=0.39218)`, clad `ZCylinder(r=0.45720)`, no gap. 17×17 `RectLattice` with 1.26 cm pitch inside a root cell bounded by ±10.71 cm planes. Boundaries are deliberately mixed: −x/+y/±z reflective, +x/−y vacuum. githubusercontent |
| Settings | 20 minimum batches, 5 inactive, 2,500 particles per batch. Tally triggers on (`trigger_active=True`, `trigger_max_batches=200`). Source: `openmc.IndependentSource(space=openmc.stats.Box(..., only_fissionable=True))`. githubusercontent |
| Plot | `openmc.Plot` + `openmc.plot_inline` (250×250 pixels, colored by material). githubusercontent |
| Tallies | (1) **Mesh tally**: `RegularMesh` 17×17, `MeshFilter` + `EnergyFilter([0, 0.625, 20e6])`, scores `fission` and `nu-fission`. (2) **Cell tally**: `CellFilter(fuel_cell)`, score `scatter`, nuclides U235 and U238. (3) **Distribcell tally**: `DistribcellFilter(moderator_cell)`, scores `absorption` and `scatter`, with a `Trigger('std_dev', 5e-5)` on absorption. githubusercontent |
| Run | `export_to_xml()` for each file, then `openmc.run()`. The stored output |

| | |
|---|---|
| | shows OpenMC 0.13.1 and NNDC HDF5 data. (githubusercontent) The trigger was never satisfied and the run went to 200 batches with k ≈ 0.68 ± 0.002. The system is strongly subcritical because of the vacuum leakage and only 1.6% enrichment. |
| Post-processing | `StatePoint` + `get_tally(name=...)`, `tally.get_values(...)` (including `value='rel_err'`), `get_pandas_dataframe(nuclides=False)`, filtering on `'energy low [eV]'`, `head` / `tail`, and plots of the mesh fission rate. (openmc) (notebook) |

**Deprecation check against 0.16.0:**

- `openmc.IndependentSource`: current. The old `openmc.Source` was split into `IndependentSource`, `FileSource` and `CompiledSource` in 0.14.0. (github +2)
- `stats.Box(only_fissionable=True)`: **deprecated since 0.15.0**. Replace it with `IndependentSource(space=Box(...), constraints={'fissionable': True})`. (openmc) This is what the official `examples/pincell/build_xml.py` does. (github) (github)
- `openmc.Plot`: **deprecated in 0.16.0** in favour of `SlicePlot` and `VoxelPlot`. (github) `Geometry.plot()` and `Universe.plot()` are simpler alternatives.
- Separate `export_to_xml()` calls plus `openmc.run()`: still supported. `openmc.Model` (with `export_to_model_xml` and `Model.run()`, which returns the path to the last statepoint) is the modern, less error-prone pattern. (openmc)
- Lattice-based pin construction: still valid. For a single pin, the official example uses `openmc.model.RectangularPrism(pitch, pitch, boundary_type='reflective')` instead of six planes. (github)
- The column names `'energy low [eV]'`, `'energy high [eV]'`, `'mean'` and `'std. dev.'` are unchanged. (openmc) (reactormc)

## 4. Theory that the notebook teaches (from docs.openmc.org/methods)

- **Eigenvalue calculations.** These use the **method of successive generations**: fission sites are banked, and N source sites are resampled for each generation. The docs state that tallies "should not be scored to" until the source has converged, which is why inactive batches exist. (openmc)
- **Shannon entropy.** It is defined as $H = -\Sigma S_i \log_2 S_i$ over a mesh. The docs note

that the source distribution generally converges more slowly than k_eff, so you should check entropy as well as k_eff. openmc

- **Tally estimators.** The docs define all three:
  - analog: $R = (1/W)\,\Sigma\, w_i$
  - collision: $R = (1/W)\,\Sigma\, w_i\,\Sigma_x/\Sigma_t$
  - track-length: $R = (1/W)\,\Sigma\, w_i\,\ell_i\,\Sigma_x$ openmc

  Track-length cannot be used with post-collision filters. `Tally.estimator=None` lets OpenMC choose, with "a preference for 'tracklength'". openmc openmc
- **Statistics.** OpenMC uses **batch statistics**, justified by the law of large numbers and the central limit theorem. github openmc Since 0.15.3, `Tally` also exposes `vov` and `figure_of_merit`. openmc

---

## Deliverable A — Jupyter notebook `pincell_pandas_demo.ipynb` (cell by cell)

> Each block below is one cell. **[MD]** marks a markdown cell and **[CODE]** a code cell.

### Cell 1 [MD]

markdown

```
# Monte Carlo Neutronics of a PWR Fuel Pin with OpenMC and pandas
This notebook builds an infinite-lattice PWR UO₂ pin cell (reflective
problem with OpenMC's continuous-energy Monte Carlo solver, and analys
pandas DataFrames. It is a modernised, heavily annotated counterpart o
`openmc-notebooks/pandas-dataframes.ipynb`, updated for OpenMC ≥ 0.16

**Learning goals:** the neutron transport equation → Monte Carlo rando
inactive/active batches and Shannon entropy → tallies, estimators and
```

### Cell 2 [MD] — Theory 1: the transport equation

markdown

```
## 1. The neutron transport equation
The angular flux ψ(r, Ω, E) (neutrons·cm⁻²·s⁻¹·sr⁻¹·eV⁻¹) obeys the st

$$\Omega\cdot\nabla\psi + \Sigma_t(\mathbf r,E)\psi = \int_0^\infty\!\

- Σ_x = N σ_x is a macroscopic cross section (cm⁻¹): atom density N (a
- φ = ∫ψ dΩ is the scalar flux, ν is neutrons per fission, χ(E) is the
- With no external source this is an **eigenvalue problem**: only the
  k_eff > 1 is supercritical, = 1 critical, < 1 subcritical.

Monte Carlo does not discretise this equation. Instead it simulates in
*expected* behaviour satisfies it exactly: distance to collision ~ Exp
∝ Σ_{t,i}, and the reaction ∝ σ_x/σ_t, all with continuous-energy nucl
```

### Cell 3 [CODE] — imports and data configuration

python

```python
import os
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import openmc

print("OpenMC version:", openmc.__version__)

# Cross-section data: OpenMC reads a cross_sections.xml index pointing
# openmc.config['cross_sections'] is initialised from $OPENMC_CROSS_SE
xs = os.environ.get("OPENMC_CROSS_SECTIONS")
if xs is None:
    # Fallback: edit this path to where you unpacked e.g. the ENDF/B-V
    xs = str(Path.home() / "nuclear_data" / "endfb-viii.0-hdf5" / "cro
openmc.config["cross_sections"] = xs
assert Path(openmc.config["cross_sections"]).exists(), "cross_sections

workdir = Path("run_pincell"); workdir.mkdir(exist_ok=True)
```

## Cell 4 [MD] — Theory 2: nuclear data

markdown

```
## 2. Nuclear data and cross sections
Evaluated libraries (ENDF/B-VIII.0, JEFF, JENDL) are processed by NJOY
file holds pointwise σ(E) at several temperatures (250–2500 K for the
probability tables, secondary angle/energy distributions, and ν(E). **
(`c_H_in_H2O`) captures the effect of molecular binding of hydrogen in
(as the original pandas notebook does) visibly distorts the thermal sp
```

## Cell 5 [CODE] — materials

python

```
# Composition follows the official examples/pincell/build_xml.py (2.4 w
uo2 = openmc.Material(name="UO2 fuel 2.4 wt%")
uo2.set_density("g/cm3", 10.29769)
uo2.add_element("U", 1.0, enrichment=2.4)    # enrichment in wt% U-235
uo2.add_element("O", 2.0)
uo2.temperature = 900                        # K, approximate fuel temp

helium = openmc.Material(name="He gap")
helium.set_density("g/cm3", 0.001598)
helium.add_element("He", 2.4044e-4)

zircaloy = openmc.Material(name="Zircaloy-4")
zircaloy.set_density("g/cm3", 6.55)
zircaloy.add_element("Sn", 0.014, "wo")
zircaloy.add_element("Fe", 0.00165, "wo")
zircaloy.add_element("Cr", 0.001, "wo")
zircaloy.add_element("Zr", 0.98335, "wo")

water = openmc.Material(name="Borated water")
water.set_density("g/cm3", 0.740582)
water.add_element("B", 4.0e-5)
water.add_element("H", 5.0e-2)
water.add_element("O", 2.4e-2)
water.add_s_alpha_beta("c_H_in_H2O")

materials = openmc.Materials([uo2, helium, zircaloy, water])
for m in materials:
    print(m.name, {k: f"{v:.3e}" for k, v in m.get_nuclide_atom_densit
```

### Cell 6 [MD]

markdown

```
`add_element` expands natural elements into isotopes using IUPAC abunda
in your cross-section library. `get_nuclide_atom_densities()` returns a
Note on temperature: OpenMC interpolates between the library temperatu
`Settings.temperature` if you need interpolation.
```

## Cell 7 [MD] — geometry theory

markdown

```
## 3. Constructive solid geometry (CSG)
Surfaces define half-spaces (-S inside, +S outside). Cells are Boolean
materials. A `RectangularPrism` with `boundary_type='reflective'` mirr
boundary. Physically this models an **infinite lattice of identical pi
result is k_∞ rather than k_eff of a finite core.
```

## Cell 8 [CODE] — geometry

python

```
pitch = 1.25984
fuel_or = openmc.ZCylinder(r=0.39218, name="Fuel OR")
clad_ir = openmc.ZCylinder(r=0.40005, name="Clad IR")
clad_or = openmc.ZCylinder(r=0.45720, name="Clad OR")
box = openmc.model.RectangularPrism(pitch, pitch, boundary_type="refle

fuel_cell = openmc.Cell(name="fuel", fill=uo2,      region=-fuel_or)
gap_cell  = openmc.Cell(name="gap",  fill=helium,   region=+fuel_or &
clad_cell = openmc.Cell(name="clad", fill=zircaloy, region=+clad_ir &
mod_cell  = openmc.Cell(name="moderator", fill=water, region=+clad_or

geometry = openmc.Geometry([fuel_cell, gap_cell, clad_cell, mod_cell])
uo2.volume = np.pi * fuel_or.r**2        # per unit height; used for v
geometry.root_universe.plot(width=(pitch, pitch), pixels=(400, 400), c
plt.title("Pin cell (colour = material)"); plt.show()
```

## Cell 9 [MD] — Theory 3: power iteration, batches and entropy

markdown

```
## 4. k-eigenvalue: power iteration, batches, Shannon entropy
OpenMC uses the **method of successive generations**. Each generation
(expected number w·νΣ_f/(kΣ_t) per collision), then resamples N sites
the stochastic analogue of power iteration, $S^{(n+1)} = \tfrac{1}{k^{
converges to the fundamental mode at a rate set by the dominance ratio

- **Inactive batches** discard results while the source converges from
  Tallying them would bias the results.
- **Active batches** each give one independent realisation of every ta
- **Shannon entropy** $H=-\sum_i S_i\log_2 S_i$ over a mesh turns the
  When H flattens out, the source has converged. The docs warn that th
- k_eff is reported as a combination of the collision, absorption and
```

### Cell 10 [CODE] — settings

python

```python
settings = openmc.Settings()
settings.run_mode = "eigenvalue"
settings.batches = 150
settings.inactive = 30
settings.particles = 10_000

ll, ur = (-pitch/2, -pitch/2, -1), (pitch/2, pitch/2, 1)
settings.source = openmc.IndependentSource(
    space=openmc.stats.Box(ll, ur),
    constraints={"fissionable": True})       # replaces the deprecated

entropy_mesh = openmc.RegularMesh()
entropy_mesh.lower_left = (-fuel_or.r, -fuel_or.r)
entropy_mesh.upper_right = (fuel_or.r, fuel_or.r)
entropy_mesh.dimension = (10, 10)
settings.entropy_mesh = entropy_mesh
settings.seed = 1                            # reproducibility
```

### Cell 11 [MD] — Theory 4: tallies and estimators

markdown

```
## 5. Tallies: filters × nuclides × scores
Every tally is $X=\int_{\text{filters}} f(\mathbf r,\Omega,E)\,\psi\,d
**Filters** set the integration domain (cell, material, energy bins, m
(f = 1), `total`, `scatter`, `absorption`, `fission` (Σ_f), `nu-fissio

Estimators:
- analog: $R_x=\frac1W\sum_{i\in A}w_i$, counting actual reactions
- collision: $R_x=\frac1W\sum_{i\in C}w_i\Sigma_x/\Sigma_t$
- track-length: $R_x=\frac1W\sum_{i\in T}w_i\ell_i\Sigma_x$, which sco

Units: results are **per source neutron**. Flux comes out as track leng
volume for cm⁻². To get absolute values (n·cm⁻²·s⁻¹), multiply by the s

Energy groups: we use a 2-group split at 0.625 eV (thermal/fast, as in
logarithmic 300-bin grid to resolve the spectrum.
```

### Cell 12 [CODE] — tallies

```
python
```

```
two_group = openmc.EnergyFilter([0.0, 0.625, 20.0e6])
fine = openmc.EnergyFilter(np.logspace(-5, np.log10(20e6), 301))   # 1
cell_f = openmc.CellFilter([fuel_cell, clad_cell, mod_cell])
mat_f = openmc.MaterialFilter([uo2])

t_spec = openmc.Tally(name="flux spectrum")
t_spec.filters = [openmc.CellFilter([fuel_cell, mod_cell]), fine]
t_spec.scores = ["flux"]

t_rr = openmc.Tally(name="fuel reaction rates")
t_rr.filters = [mat_f, two_group]
t_rr.nuclides = ["U235", "U238", "O16"]
t_rr.scores = ["total", "scatter", "absorption", "fission", "nu-fissio

t_cells = openmc.Tally(name="cell 2-group")
t_cells.filters = [cell_f, two_group]
t_cells.scores = ["flux", "absorption"]

mesh = openmc.RegularMesh()
mesh.lower_left, mesh.upper_right, mesh.dimension = (-pitch/2, -pitch/
t_mesh = openmc.Tally(name="mesh")
t_mesh.filters = [openmc.MeshFilter(mesh), two_group]
t_mesh.scores = ["flux", "fission"]

tallies = openmc.Tallies([t_spec, t_rr, t_cells, t_mesh])
```

### Cell 13 [CODE] — build and run the Model

python

```
model = openmc.Model(geometry=geometry, materials=materials, settings=
sp_path = model.run(cwd=workdir)          # exports model.xml, runs op
print("Statepoint:", sp_path)
```

### Cell 14 [MD]

markdown

`Model.run()` writes `model.xml` in `run_pincell/`, runs the `openmc` e
`statepoint.150.h5`. Equivalent legacy workflow: `materials.export_to_
openmc.run()`. A `summary.h5` file is also written, so cells and mater
DataFrames.

## Cell 15 [CODE] — convergence diagnostics

python

```python
with openmc.StatePoint(sp_path) as sp:
    keff = sp.keff                                  # uncertainties.UFloat: c
    k_gen = np.array(sp.k_generation)
    entropy = np.array(sp.entropy)
    n_inactive = sp.n_inactive
    spec = sp.get_tally(name="flux spectrum")
    rr = sp.get_tally(name="fuel reaction rates")
    cells = sp.get_tally(name="cell 2-group")
    mtal = sp.get_tally(name="mesh")
    df_spec = spec.get_pandas_dataframe()
    df_rr = rr.get_pandas_dataframe()
    df_cells = cells.get_pandas_dataframe()
    df_mesh = mtal.get_pandas_dataframe()

print(f"k_inf = {keff.nominal_value:.5f} ± {keff.std_dev:.5f}  ({1e5*k
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(k_gen); ax[0].axvline(n_inactive, ls="--", c="k"); ax[0].se
ax[1].plot(entropy); ax[1].axvline(n_inactive, ls="--", c="k"); ax[1].
plt.tight_layout(); plt.show()
```

## Cell 16 [MD] — interpretation

markdown

```
**Reading these plots.** The generation-wise k fluctuates around the m
per batch, while the uncertainty of the *mean* shrinks as $1/\sqrt{N\,
plateau well before the dashed line (the end of the inactive batches).
`settings.inactive`. Expect k_∞ ≈ 1.2–1.3 for 2.4% fuel in an infinite
libraries or settings is physics, not a bug.
Caveat: batch-to-batch correlation in power iteration makes the report
**underestimate** the true uncertainty. This is a well-known limitatio
```

## Cell 17 [MD] — anatomy of the DataFrame

markdown

```
## 6. Anatomy of a tally DataFrame
`Tally.get_pandas_dataframe()` returns one row per (filter-bin combina
for each filter (e.g. `cell`, `material`, `energy low [eV]`, `energy hi
`nuclide`, `score`, `mean` and `std. dev.`. This "long" (tidy) format
`pivot_table` expect.
```

## Cell 18 [CODE]

python

```
pd.set_option("display.float_format", "{:.4e}".format)
df_rr["rel. err."] = df_rr["std. dev."] / df_rr["mean"]
display(df_rr.head(12))
print(df_rr.columns.tolist())
```

## Cell 19 [MD] — filtering, grouping and pivoting

markdown

```
### Filtering, grouping, pivoting
Below we (1) select thermal fission, (2) pivot to a nuclide × score ta
- η (neutrons per absorption in fuel) = νΣ_f / Σ_a
- the thermal fission fraction

Uncertainties of ratios are propagated to first order, assuming no cor
For correlated arithmetic, use `Tally` arithmetic (`t1 / t2`) instead.
```

## Cell 20 [CODE]

python

```python
thermal = df_rr[df_rr["energy high [eV]"] <= 0.625]
print(thermal.query("score == 'fission'")[["nuclide", "mean", "rel. er

pivot = df_rr.pivot_table(index="nuclide", columns="score", values="mea
display(pivot)

tot = df_rr.groupby("score")[["mean"]].sum()
eta = tot.loc["nu-fission", "mean"] / tot.loc["absorption", "mean"]
fis = df_rr.query("score=='fission'").groupby("energy high [eV]")["mean
print(f"eta (fuel) = {eta:.4f};  thermal fission fraction = {fis.iloc[0
print("U-238 share of fuel absorption:", pivot.loc["U238", "absorption"
```

## Cell 21 [MD]

markdown

```markdown
**Interpretation.** Most fissions are in U-235 and are thermal. U-238 c
large share of absorption (resonance capture), which is the origin of t
four-factor formula $k_\infty=\eta f p \varepsilon$.
```

## Cell 22 [CODE] — flux spectrum

python

```python
df_spec["E_mid"] = np.sqrt(df_spec["energy low [eV]"].clip(lower=1e-5)
df_spec["lethargy width"] = np.log(df_spec["energy high [eV]"] / df_spe
df_spec["flux/u"] = df_spec["mean"] / df_spec["lethargy width"]
fig, ax = plt.subplots(figsize=(8, 5))
for cell_id, g in df_spec.groupby("cell"):
    ax.step(g["E_mid"], g["flux/u"], where="mid", label=f"cell {cell_id
    ax.fill_between(g["E_mid"], g["flux/u"] - g["std. dev."]/g["lethargy
                    g["flux/u"] + g["std. dev."]/g["lethargy width"], s
ax.set(xscale="log", xlabel="Energy [eV]", ylabel="Flux per unit lethar
ax.legend(); plt.show()
```

## Cell 23 [MD]

markdown

```
**Physics in the spectrum.**
- A Maxwellian thermal peak near 0.025–0.1 eV, larger in the moderator
- A 1/E slowing-down region in between, where flux per unit lethargy is
- Sharp dips at the U-238 resonances (6.67, 20.9, 36.7 eV …) inside the
- The fission-spectrum hump near 1–2 MeV.

Plotting per unit lethargy (u = ln(E₀/E)) is the convention because bin
"visible" on a log axis.
```

## Cell 24 [CODE] — relative errors and the mesh map

python

```
df_cells["rel. err."] = df_cells["std. dev."] / df_cells["mean"]
display(df_cells.pivot_table(index="cell", columns=["score", "energy hi

fis = df_mesh.query("score=='fission' and `energy high [eV]` <= 0.625"
img = fis["mean"].to_numpy().reshape(50, 50)       # filter ordering: x
plt.imshow(img, origin="lower", extent=[-pitch/2, pitch/2]*2); plt.colo
plt.title("Thermal fission rate (note rim enhancement/self-shielding)"
plt.hist((df_mesh["std. dev."]/df_mesh["mean"]).replace(np.inf, np.nan)
plt.xlabel("relative error"); plt.ylabel("# mesh bins"); plt.show()
```

## Cell 25 [MD] — uncertainties and convergence

markdown

```
## 7. Uncertainties and statistical convergence
Each active batch gives one realisation $x_i$. OpenMC reports $\bar x$
$s_{\bar x}=\sqrt{\frac{1}{n(n-1)}\sum(x_i-\bar x)^2}$. By the central
Rules of thumb:
- Relative error < 5% for local tallies and < 1% for integral quantiti
- Mesh bins outside the fuel have **zero** fission (mean 0, rel. err.
- Error ∝ 1/√(total histories), so halving σ costs 4× the CPU time.
- The figure of merit FOM = 1/(R²T) should be roughly constant if the
  (`Tally.figure_of_merit`, available since 0.15.3).
- Tally triggers (`openmc.Trigger`) can automate "run until σ < thresh
  Its 5e-5 absolute threshold was never met.
```

### Cell 26 [MD] — exercises

markdown

```
## 8. Exercises
1. Vary the enrichment (1.6–5%) and plot k_∞ ± σ.
2. Remove `c_H_in_H2O` and compare the spectra.
3. Change the boron concentration and compute the boron worth in pcm.
4. Switch to vacuum on one side (as in the original notebook) and watc
5. Force `t_rr.estimator = 'collision'` and compare the variance with
```

---

## Deliverable B — `environment.yml` and data setup

yaml

```yaml
name: openmc-pincell
channels:
  - conda-forge
dependencies:
  - python=3.12          # OpenMC 0.16 requires >=3.12
  - openmc=0.16.0        # pin for reproducibility (non-MPI build is f
  - numpy
  - pandas
  - scipy
  - matplotlib
  - h5py
  - uncertainties
  - lxml
  - jupyterlab
  - ipykernel
  - pip
  - pip:
      - openmc_data      # optional: download_endf helper (third-party
```

**Data setup (Linux/macOS/WSL2):**

bash

```bash
conda config --set channel_priority strict
conda env create -f environment.yml && conda activate openmc-pincell
mkdir -p ~/nuclear_data && cd ~/nuclear_data
# Download the ENDF/B-VIII.0 .tar.xz linked at https://openmc.org/data/
tar -xJf <downloaded-file>.tar.xz
export OPENMC_CROSS_SECTIONS=~/nuclear_data/endfb-viii.0-hdf5/cross_se
# Persist it inside the env:
conda env config vars set OPENMC_CROSS_SECTIONS=$OPENMC_CROSS_SECTIONS
python -m ipykernel install --user --name openmc-pincell
```

Alternatively, after installing `openmc_data`, run `download_endf -r b8.0` (check `download_endf --help` for your version).

---

## Deliverable C — `README.md`

markdown

````
# OpenMC Pin-Cell Demonstrator: Monte Carlo Neutronics + pandas

A small, reproducible demonstrator of continuous-energy Monte Carlo ne
[OpenMC](https://openmc.org) Python API (v0.16). It models an infinite
(Zircaloy-4 cladding, borated light water, reflective boundaries), sol
analyses flux spectra, reaction rates and uncertainties with pandas.

## Repository structure
```
.
├── README.md
├── environment.yml
├── notebooks/
│   └── pincell_pandas_demo.ipynb
├── run_pincell/            # created at runtime (model.xml, statepoin
└── LICENSE
```

## Installation
1. Install Miniforge/Miniconda. Linux and macOS are supported natively
   (`docker run openmc/openmc:latest`).
2. `conda env create -f environment.yml && conda activate openmc-pince
3. Download nuclear data: the ENDF/B-VIII.0 HDF5 library from https://
4. `export OPENMC_CROSS_SECTIONS=/abs/path/to/cross_sections.xml`, or
   in the notebook.

## Running
```bash
jupyter lab notebooks/pincell_pandas_demo.ipynb
```
Run all cells (~1–3 min on a laptop with 10k particles × 150 batches).
exact values depend on the data library.

## Theory summary
**Transport equation (k-eigenvalue form):**

$$\boldsymbol\Omega\cdot\nabla\psi+\Sigma_t\psi=\int\!\!\int\Sigma_s(E
````

```
**Macroscopic cross section:** $\Sigma_x = N\sigma_x$. **Free-flight sa

**Power iteration (method of successive generations):** $S^{(n+1)}=\fra
$k^{(n+1)}\approx \frac{\text{fission neutrons in gen } n+1}{\text{sou

**Shannon entropy:** $H=-\sum_i S_i\log_2 S_i$, used to choose the numb

**Tally estimators:**
- analog: $R_x=\frac1W\sum_{A}w_i$
- collision: $R_x=\frac1W\sum_{C}w_i\frac{\Sigma_x}{\Sigma_t}$
- track-length: $R_x=\frac1W\sum_{T}w_i\ell_i\Sigma_x$

**Batch statistics:** $\bar x=\frac1n\sum x_i,\quad s_{\bar x}=\sqrt{\
relative error $R=s_{\bar x}/\bar x$, and $\mathrm{FOM}=1/(R^2T)$.

**Four-factor formula (infinite medium):** $k_\infty=\eta f p\varepsil

## API notes (differences from the official pandas-dataframes notebook
- Uses `openmc.Model` + `Model.run()` instead of separate `export_to_xm
- Uses `IndependentSource(..., constraints={'fissionable': True})`. `Bo
- Uses `Universe.plot()` instead of `openmc.Plot`/`plot_inline`. `Plot
- Single pin with `openmc.model.RectangularPrism(..., boundary_type='re
- Adds `c_H_in_H2O` thermal scattering and a He gap, following `openmc/

## References
- OpenMC documentation (stable): https://docs.openmc.org/en/stable/. Se
- OpenMC v0.16.0 release notes: https://github.com/openmc-dev/openmc/re
- Example notebooks: https://github.com/openmc-dev/openmc-notebooks (pa
- Nuclear data: https://openmc.org/data/ ; scripts: https://github.com/
- P. K. Romano et al., "OpenMC: A state-of-the-art Monte Carlo code fo
- P. K. Romano, B. Forget, "The OpenMC Monte Carlo particle transport c
- J. Lieberoth, "A Monte Carlo technique to solve the static eigenvalue
- F. B. Brown, "On the use of Shannon entropy of the fission distributi
- J. J. Duderstadt, L. J. Hamilton, *Nuclear Reactor Analysis*, Wiley
- D. L. Brown et al., "ENDF/B-VIII.0," *Nucl. Data Sheets* 148, 1-142
```

## Recommendations

1. **Pin everything.** Pin `openmc=0.16.0`, the data library and version (ENDF/B-VIII.0 from openmc.org) and `settings.seed`, so that published results reproduce bit-for-bit on the same build.
2. **Add CI.** Run the notebook with a tiny particle count (e.g. 500 particles × 20 batches) in GitHub Actions. Use `nbconvert --execute` and cache the data download (~GBs). If downloads are too heavy, cache only the nuclides you need by trimming a copy of `cross_sections.xml`.
3. **Commit no outputs.** Keep `*.h5` and `*.xml` out of git and add `nbstripout`.
4. **For the multimodal/ML angle,** the tidy DataFrames (spectra, mesh maps) export directly to Parquet. Parametric sweeps over enrichment, boron and temperature make a good small surrogate-modelling dataset.

## Caveats

- **Unverified.** I could not run OpenMC here, so the notebook code has not been executed. Every call it uses is documented for the stable API, but check column names on first run, particularly the mesh filter columns, whose labels changed across versions. The numerical expectations ($k_\infty \approx 1.2$–$1.3$) are physics-based estimates, not measured results.
- **Third-party tooling.** `openmc_data` is maintained outside openmc-dev, and its CLI flags may change.
- **Apple-silicon note.** The Quick Install Guide's `--platform osx-64` note appears outdated relative to the osx-arm64 builds on conda-forge. (anaconda) (openmc)
- **Mixed-version stored output.** The stored output of the official pandas notebook comes from 0.13.1, while its source code was later partly updated (`IndependentSource`). (githubusercontent) Its printed results therefore don't exactly match what the current code produces.